ALPE FORMACIÓN · SANTANDER

Edición 2026 · PHP 8.x

CURSO DE PHP · CLASE 23

# Arquitectura MVC y front controller

- MVC: el restaurante bien organizado
- El problema de los archivos sueltos
- La estructura del proyecto: solo public/ es visible
- El front controller y el .htaccess
- El recorrido de una petición
- El controlador
- Las vistas y el layout
- El modelo



**Ramón Abramo**

+25 años de experiencia docente · Santander, Cantabria

ALPE

FORMACIÓN PROFESIONAL

# 🧭 Contenido de la clase

Empieza una de las partes más bonitas del curso: **vamos a construir nuestro propio *framework***. Paso a paso, clase a clase, iremos montando las mismas piezas que tiene Laravel por dentro: un único punto de entrada, un enrutador, controladores, vistas, modelos, middleware...

¿Para qué, si Laravel ya existe? Por lo mismo que en una autoescuela os enseñan qué hace el embrague: cuando lleguéis a Laravel, nada os parecerá magia. Sabréis qué pasa por debajo, y eso os convertirá en mejores programadores de Laravel.

Hoy ponemos los cimientos: el patrón **MVC** y el **front controller**.

•
👌 Cómo vamos a trabajar



1
📘 MVC: el restaurante bien organizado



2
🤔 El problema de los archivos sueltos



3
⚠️ La estructura del proyecto: solo public/ es visible



4
🔍 El front controller y el .htaccess



5
🚀 El recorrido de una petición



6
👍 El controlador



7
👌 Las vistas y el layout



8
🔍 El modelo



9
😮 Lo que todavía falla



10
🚀 Ejercicios



11
👋 Lo que tenéis que llevaros de hoy





CLASE 23

## 👌 Cómo vamos a trabajar

Durante las próximas cinco clases trabajaremos en **un único proyecto**, `academia-mvc`, que irá creciendo. Cada clase tiene en su carpeta `proyecto/` la versión de ese día, completa y funcionando.

1. En la raíz de Laragon, cread la carpeta `academia-mvc` y copiad dentro el contenido de `proyecto/`.
2. Usamos la base de datos de las clases 16 y 21, `academia_crud`. Si no la tenéis, ejecutad `base-de-datos.sql` en HeidiSQL.
3. En la Terminal de Laragon, dentro de la carpeta: `composer install` (solo genera la autocarga).
4. Pulsad **Recargar** y abrid `http://academia-mvc.test`.



💡 ¿No usáis Laragon o algo no va? Desde la carpeta del proyecto, `composer servidor` arranca el servidor integrado de PHP en `http://localhost:8000`, ya configurado para el front controller.



CLASE 23 · PUNTO 1

## 📘 MVC: el restaurante bien organizado

**MVC** son las siglas de **Modelo, Vista y Controlador**: tres piezas con tareas separadas que no se mezclan [1]. Volvamos a nuestro restaurante de la clase 1:

| Pieza | Qué hace | En el restaurante |
|---|---|---|
| **Controlador** | Recibe la petición, decide qué hacer, pide los datos y elige cómo presentarlos | El **camarero** y el jefe de sala: toman la comanda y la organizan |
| **Modelo** | Los datos y sus reglas: la base de datos, las validaciones, los cálculos | La **cocina** y la **despensa**: donde están los ingredientes y las recetas |
| **Vista** | La presentación: el HTML que ve el usuario | El **emplatado**: cómo llega el plato a la mesa |

La regla de oro: **cada pieza solo hace lo suyo**. La vista no consulta la base de datos. El modelo no escribe HTML. El controlador no hace cálculos de negocio: coordina.



🎯 Ya conocéis dos de las tres piezas. El **modelo** es lo de la clase 21 (la entidad `Curso` y el `CursoRepositorio`). Las **vistas** son las plantillas con sintaxis alternativa que usáis desde la clase 8. Y el **controlador** ya lo asomamos en la clase 12, cuando partimos la inscripción en `inscripcion.php` (decide) e `inscripcion.vista.php` (pinta). Hoy lo convertimos en una pieza de verdad y lo organizamos todo.



CLASE 23 · PUNTO 2

## 🤔 El problema de los archivos sueltos

En la clase 21 cada página era un archivo: `index.php`, `crear.php`, `editar.php`... Funciona, pero tiene problemas que crecen con la web:

| Problema | Ejemplo |
|---|---|
| Cada archivo repite el arranque | El `require` de `inicio.php`, la sesión, la conexión... en cada página |
| Las direcciones dependen de los archivos | `editar.php?id=5` en lugar de `/cursos/5/editar` |
| Todo lo que está en la carpeta se puede pedir desde el navegador | `config/config.php`, `src/…`, `base-de-datos.sql` |
| No hay un sitio único para errores, seguridad o registros | Hay que acordarse en cada archivo |

La solución es el **front controller** (controlador frontal): **una única puerta de entrada** por la que pasan todas las peticiones de la web. Como la recepción de un hotel: da igual a qué habitación vayáis, siempre pasáis por recepción [2].



CLASE 23 · PUNTO 3

## ⚠️ La estructura del proyecto: solo public/ es visible

Esquema

```
academia-mvc/
├── .htaccess                ← si Apache apunta aquí, todo va a public/
├── composer.json
├── config/config.php        ← configuración          ┐
├── src/                     ← App\ (nuestras clases)  │  INVISIBLES
│   ├── Controladores/       ← los controladores       │  desde el
│   ├── Modelos/             ← Curso                   │  navegador
│   ├── Repositorios/        ← CursoRepositorio        │
│   ├── BaseDatos/           ← Conexion                │
│   └── funciones.php        ← e(), euros(), vista()   │
├── vistas/                  ← las plantillas HTML     ┘
│   ├── layout.php
│   ├── cursos/index.php
│   ├── cursos/ver.php
│   └── errores/404.php
└── public/                  ← lo ÚNICO que se ve desde el navegador
    ├── .htaccess
    ├── index.php            ← el front controller
    └── estilos.css
```

El servidor web solo debe servir la carpeta `public/`. Ahí está el `index.php` y los archivos que el navegador necesita de verdad (CSS, imágenes, JavaScript). El resto del proyecto queda **fuera del alcance** del navegador: nadie puede pedir vuestro `config.php` o vuestro SQL. Es exactamente la estructura de Laravel.



💡 En Laragon, el *virtual host* `academia-mvc.test` apunta a la carpeta del proyecto. Por eso hay un `.htaccess` en la raíz que reenvía todo a `public/`. En un servidor de verdad, se configura directamente `public/` como carpeta raíz de la web.



CLASE 23 · PUNTO 4

## 🔍 El front controller y el .htaccess

¿Cómo conseguimos que `/cursos/5`, que **no es ningún archivo**, llegue a `index.php`? Con unas reglas para Apache en el archivo `public/.htaccess` [3]:

Esquema

```
RewriteEngine On
RewriteCond %{REQUEST_FILENAME} !-f
RewriteCond %{REQUEST_FILENAME} !-d
RewriteRule ^ index.php [QSA,L]
```

Leedlo así: «si lo que se pide **no** es un archivo (`!-f`) **ni** una carpeta (`!-d`) que existan, entrégaselo a `index.php`». Así, `/estilos.css` se sirve tal cual (existe), pero `/cursos/5` acaba en el front controller. `QSA` conserva lo que va detrás del `?` (por ejemplo, `?q=php`).

Y el front controller lee la dirección pedida y decide a quién llamar:

index.php — php-paso-a-paso/clase23 — Visual Studio Code

index.php

```php
<?php
declare(strict_types=1);

require_once __DIR__ . "/../vendor/autoload.php";
require_once __DIR__ . "/../config/config.php";

use App\Controladores\CursosControlador;

$metodo = $_SERVER["REQUEST_METHOD"];
$ruta = parse_url($_SERVER["REQUEST_URI"], PHP_URL_PATH);
$ruta = rtrim($ruta, "/") ?: "/";      // "/cursos/" y "/cursos" son la misma ruta

$controlador = new CursosControlador();

if ($metodo === "GET" && ($ruta === "/" || $ruta === "/cursos")) {
    $html = $controlador->index();
} elseif ($metodo === "GET" && preg_match("#^/cursos/(\d+)$#", $ruta, $partes)) {
    $html = $controlador->ver((int) $partes[1]);
} else {
    http_response_code(404);
    $html = vista("errores/404", ["titulo" => "Página no encontrada"]);
}

echo $html;
```

| Pieza | Qué hace |
|---|---|
| `$_SERVER["REQUEST_METHOD"]` | El método: `GET`, `POST`… |
| `$_SERVER["REQUEST_URI"]` | Lo que se ha pedido: `/cursos/5?q=php` |
| `parse_url(..., PHP_URL_PATH)` | Se queda solo con la ruta: `/cursos/5` |
| `preg_match("#^/cursos/(\d+)$#", ...)` | ¿La ruta es `/cursos/` seguido de un número? El número queda en `$partes[1]` |

📄 Archivos: `public/index.php` y `public/.htaccess`



💡 La expresión regular `#^/cursos/(\d+)$#` significa: empieza ( `^` ) por `/cursos/` , sigue uno o más dígitos ( `\d+` ), que **guardamos** gracias a los paréntesis, y ahí termina ( `$` ).



CLASE 23 · PUNTO 5

## 🚀 El recorrido de una petición

Esquema

```
Navegador: GET /cursos/2
     │
     ▼
Apache + .htaccess ──► public/index.php            (FRONT CONTROLLER)
                              │  ¿qué ruta es? → ver con id 2
                              ▼
                    CursosControlador::ver(2)      (CONTROLADOR)
                              │                    │
          pide el curso       │                    │ elige la vista y le pasa el curso
                              ▼                    ▼
         CursoRepositorio → Curso      vistas/cursos/ver.php   (MODELO y VISTA)
                                            │ dentro de vistas/layout.php
                                            ▼
                                  HTML de vuelta al navegador
```

Memorizad este dibujo. **Toda** petición seguirá este camino, en nuestro MVC y en Laravel. Lo único que irá cambiando estas clases es lo bien hecha que esté cada pieza.



CLASE 23 · PUNTO 6

## 👍 El controlador

Un controlador es una clase con un método por cada «acción» de la web. Recibe la petición, pide los datos al modelo y devuelve la vista. Nada más [1].

CursosControlador.php — php-paso-a-paso/clase23 — Visual Studio Code

CursosControlador.php

```php
<?php
declare(strict_types=1);

namespace App\Controladores;

use App\BaseDatos\Conexion;
use App\Repositorios\CursoRepositorio;

class CursosControlador
{
    private CursoRepositorio $cursos;

    public function __construct()
    {
        $this->cursos = new CursoRepositorio(Conexion::obtener());
    }

    // GET /cursos
    public function index(): string
    {
        $busqueda = trim($_GET["q"] ?? "");
        $cursos = $this->cursos->buscar($busqueda, 1, 100);

        return vista("cursos/index", ["titulo" => "Cursos", "cursos" => $cursos, "busqueda"
=> $busqueda]);
    }

    // GET /cursos/{id}
    public function ver(int $id): string
    {
        $curso = $this->cursos->buscarPorId($id);
        if ($curso === null) {
            http_response_code(404);
            return vista("errores/404", ["titulo" => "Curso no encontrado"]);
        }
        return vista("cursos/ver", ["titulo" => $curso->titulo, "curso" => $curso]);
    }
}
```

Fijaos en lo cortos que son los métodos: **no hay ni SQL ni HTML**. El SQL está en el repositorio y el HTML, en las vistas.

📄 Archivo: `src/Controladores/CursosControlador.php`



CLASE 23 · PUNTO 7

# 👌 Las vistas y el layout

Las vistas son plantillas **sin lógica**: solo HTML, `<?= ?>` y algún `foreach` o `if` para mostrar. Como cada página comparte cabecera y pie, usamos un **layout** (plantilla común), igual que la cabecera y el pie de la clase 14, pero al revés: la vista se mete **dentro** del layout.

funciones.php — php-paso-a-paso/clase23 — Visual Studio Code

funciones.php

```php
<?php
function vista(string $nombre, array $datos = []): string
{
    extract($datos);

    ob_start();
    require __DIR__ . "/../vistas/{$nombre}.php";
    $contenido = ob_get_clean();

    ob_start();
    require __DIR__ . "/../vistas/layout.php";
    return ob_get_clean();
}
```

Dos funciones nuevas hacen el truco [4]:

| Función | Qué hace |
|---|---|
| `extract($datos)` | Convierte cada clave del array en una variable: `["cursos" ⇒ …]` pasa a ser `$cursos` dentro de la vista |
| `ob_start()` / `ob_get_clean()` | «Graba» todo lo que se imprime en lugar de enviarlo al navegador, y lo devuelve como texto |

Así, la vista se genera y se guarda en `$contenido`, y después el layout la imprime en su sitio con `<?= $contenido ?>`. Como en un marco de fotos: el layout es el marco y cada vista, la foto.

📄 Archivos: `src/funciones.php`, `vistas/layout.php` y `vistas/cursos/index.php`



⚠️ Fijaos en los enlaces: ahora empiezan por `/` (`/cursos/5`, `/estilos.css`). Con direcciones como `/cursos/5`, un enlace relativo como `estilos.css` se buscaría en `/cursos/estilos.css` y no existiría.



CLASE 23 · PUNTO 8

## 🔍 El modelo

Hoy no escribimos ni una línea de modelo: reutilizamos el `Curso` y el `CursoRepositorio` de la clase 21, **sin cambiar nada**. Esa es precisamente la gracia de haber separado bien las piezas. Las clases que no saben de dónde las llaman se pueden llevar a cualquier proyecto.

📄 Carpetas: `src/Modelos/` , `src/Repositorios/` y `src/BaseDatos/`



💡 En la clase 25 cambiaremos el repositorio por un **modelo base** parecido a Eloquent, el de Laravel, para que veáis las dos formas habituales de organizar el modelo.



CLASE 23 · PUNTO 9

## 😮 Lo que todavía falla

Nuestro MVC funciona, pero tiene puntos débiles que arreglaremos en las próximas clases:

| Problema | Lo arreglamos en… |
|---|---|
| Las rutas son una cadena de `if/elseif` que crecerá sin fin | Clase 24: un **Router** |
| Usamos `$_GET` , `$_SERVER` y `echo` directamente | Clase 24: clases **Petición** y **Respuesta** |
| Si algo falla, el visitante ve el error de PHP | Clase 24: el **manejador global** de la clase 22 |
| Solo leemos: no podemos crear, editar ni borrar | Clase 25: el CRUD completo y un **controlador base** |
| Cada controlador crea sus propios objetos con `new` | Clase 26: un **contenedor de dependencias** |
| No hay login ni protección CSRF | Clase 26: **middleware** y autenticación |



CLASE 23 · PUNTO 10

## 🚀 Ejercicios

Las soluciones se copian dentro del proyecto, respetando sus carpetas.

- ☐ **01 · Una página nueva.** Añadid la ruta `GET /acerca` con un controlador nuevo, `PaginasControlador`, y su vista `vistas/paginas/acerca.php`. → `PaginasControlador.php`, `acerca.php` y `public/index.php`
- ☐ **02 · Cursos completos.** Añadid `GET /cursos/completos`, que muestre solo los cursos sin plazas, reutilizando la vista del listado. ¿En qué orden tienen que ir las rutas en el front controller? → `CursosControlador.php`
- ☐ **03 · Paginación.** Recuperad la paginación de la clase 16 en el listado (`?pagina=2`), usando `contar()` y la constante `CURSOS_POR_PAGINA`. → `CursosControlador.php` y `vistas/cursos/index.php`



🏆 **Reto:** intentad abrir `http://academia-mvc.test/../config/config.php` o `http://academia-mvc.test/src/funciones.php`. ¿Qué pasa? ¿Por qué? Explicadlo con lo que sabéis del `.htaccess` y de la carpeta `public/`.



CLASE 23 · PUNTO 11

## 👋 Lo que tenéis que llevaros de hoy

| Idea | En una frase |
|---|---|
| MVC | Modelo (datos y reglas), Vista (HTML) y Controlador (coordina) |
| Front controller | Una sola puerta de entrada: `public/index.php` |
| `.htaccess` | Lo que no es un archivo real se entrega a `index.php` |
| `public/` | Lo único visible desde el navegador; el resto del proyecto queda protegido |
| Rutas bonitas | `/cursos/5` en lugar de `ver.php?id=5` |
| Controlador | Un método por acción, sin SQL ni HTML |
| Vistas y layout | `extract()` + `ob_start()`: la vista se mete dentro del layout |
| Modelo | Reutilizamos entidad y repositorio de la clase 21 sin tocar nada |

# 📚 Fuentes y documentación oficial

1. MDN Web Docs: MVC
2. Martin Fowler: Front Controller
3. Apache HTTP Server: mod_rewrite
4. Manual de PHP: control de salida (ob_start)